

ES98A Assignment 4

Author: Jabir Hussain (2007942)

Exercise 4.1.

For this exercise, we have the normal random variable $x \sim \mathcal{N}(m, \sigma^2)$, $m \in \mathbb{R}$, $\sigma^2 \geq 0$. We define $y := e^x$. Let $z \sim \mathcal{N}(0, 1)$ such that z has the same distribution as $m + \sigma z$. In turn, y has the same distribution as $e^{m+\sigma z}$. Consider the Hermite polynomial chaos expansion

$$y = \sum_{k \geq 0} y_k \text{He}_k(z).$$

(a) Show that

$$y_k = \frac{e^{m+\sigma^2/2}}{k!} \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \text{He}_k(w + \sigma) e^{-w^2/2} dw.$$

(b) Using the translation formula

$$\text{He}_n(x + y) = \sum_{j=0}^n \binom{n}{j} x^{n-j} \text{He}_j(y)$$

show that

$$y_k = \frac{e^{m+\sigma^2/2} \sigma^k}{k!}.$$

(c) Use the polynomial chaos expansion to compute the mean and variance of y . Note well that $\mathbb{E}[y] \neq e^m$ and $\text{Var}[y] \neq e^{\sigma^2}$.

Exercise 4.1(a)

We denote y as a function of the standard normal variable z where

$$y(z) = e^{m+\sigma z}.$$

Consider a Hermite polynomial chaos expansion in z :

$$y(z) = \sum_{k \geq 0} y_k \text{He}_k(z).$$

For Hermite polynomials, the normalisation constant is given by $\gamma_k = k!$. This means the orthogonality relation takes the form The key property from the lectures is the orthogonality relation

$$\mathbb{E}[\text{He}_j(z) \text{He}_k(z)] = k! \delta_{jk},$$

Using the general formula for chaos coefficients, we can write the expression in the form

$$y(z) = \sum_{k \geq 0} y_k \psi_k(z)$$

with orthogonality

$$\mathbb{E}[\psi_j(z) \psi_k(z)] = \gamma_k \delta_{jk}.$$

The coefficient is given by

$$y_k = \frac{1}{\gamma_k} \mathbb{E}[y(z) \psi_k(z)] = \gamma_k \delta_{jk} \in \mathbb{R}$$

where $\psi_k(z) = \text{He}_k(z)$, $\gamma_k = \mathbb{E}[\text{He}_k(z)^2] = k!$

Substituting these expressions in gives

$$y_k = \frac{1}{\mathbb{E}[\text{He}_k(z)^2]} \mathbb{E}[y(z) \text{He}_k(z)] = \frac{1}{k!} \mathbb{E}[y(z) \text{He}_k(z)].$$

As $y(z) = e^{m+\sigma z}$, it follows that

$$y_k = \frac{1}{k!} \mathbb{E}[e^{m+\sigma z} \text{He}_k(z)].$$

We can write this expectation as an integral. The density of $z \sim \mathcal{N}(0, 1)$ is given by

$$F(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}.$$

For any suitable function g ,

$$\mathbb{E}[g(z)] = \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} g(z) e^{-\frac{z^2}{2}} dz.$$

For $g(z) = e^{m+\sigma z} \text{He}_k(z)$, we get

$$y_k = \frac{1}{k!} \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} e^{m+\sigma z} \text{He}_k(z) e^{-\frac{z^2}{2}} dz.$$

Focusing only on the exponential terms:

$$e^{m+\sigma z} e^{-\frac{z^2}{2}} = e^m e^{\sigma z - \frac{z^2}{2}}$$

More specifically, the second e terms exponent:

$$\begin{aligned} \sigma z - \frac{z^2}{2} &= -\frac{1}{2}(z^2 - 2\sigma z) \\ z^2 - 2\sigma z &= (z - \sigma)^2 - \sigma^2 \implies \sigma z - \frac{z^2}{2} = -\frac{1}{2}((z - \sigma)^2 - \sigma^2) = -\frac{(z - \sigma)^2}{2} + \frac{\sigma^2}{2} \end{aligned}$$

Thus we get

$$e^{\sigma z - z^2/2} = e^{\frac{\sigma^2}{2}} e^{-\frac{(z-\sigma)^2}{2}}.$$

Substituting this into our equation for y_k gives

$$\begin{aligned} y_k &= \frac{1}{k!} \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \text{He}_k(z) e^{m+\sigma z - z^2/2} dz \\ &= \frac{1}{k!} \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \text{He}_k(z) e^{m+\sigma^2/2} e^{-(z-\sigma)^2/2} dz \\ &= \frac{e^{m+\sigma^2/2}}{k!} \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \text{He}_k(z) e^{-(z-\sigma)^2/2} dz. \end{aligned}$$

We define $w = z - \sigma \implies z = w + \sigma \implies dz = dw$. In turn, $\text{He}_k(w + \sigma)$, $e^{-(z-\sigma)^2/2} = e^{-\frac{w^2}{2}}$. Substituting these into the expression for y_k , we get

$$y_k = \frac{e^{m+\sigma^2/2}}{k!} \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \text{He}_k(w + \sigma) e^{-w^2/2} dw$$

Exercise 4.1(b)

The Hermite translation is given by the formula

$$\text{He}_n(x + y) = \sum_{j=0}^n \binom{n}{j} x^{n-j} \text{He}_j(y)$$

Substituting $n = k$, $x = \sigma$, $y = w$ gives

$$\text{He}_k(w + \sigma) = \sum_{j=0}^k \binom{k}{j} \sigma^{k-j} \text{He}_j(w)$$

Substituting this expression into our equation for y_k gives

$$y_k = \frac{e^{m+\sigma^2/2}}{k!} \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \left(\sum_{j=0}^k \binom{k}{j} \sigma^{k-j} \text{He}_j(w) \right) e^{-w^2/2} dw$$

$$= \frac{e^{m+\sigma^2/2}}{k!} \sum_{j=0}^k \binom{k}{j} \sigma^{k-j} \underbrace{\left(\frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \text{He}_j(w) e^{-w^2/2} dw \right)}_{\mathbb{E}[\text{He}_j(z)]}.$$

It is known that $\text{He}_0(z) = 1$ and $\mathbb{E}[\text{He}_j(z)\text{He}_0(z)] = j!\delta_{j0}$. As $\text{He}_0(z) = 1$, the left hand side is just $\mathbb{E}[\text{He}_j(z)] \implies \mathbb{E}[\text{He}_j(z)] = j!\delta_{j0}$.

For $j = 0$, $\mathbb{E}[\text{He}_0(z)] = 0! \cdot 1 = 1$ and for $j \geq 1$, $\mathbb{E}[\text{He}_j(z)] = j! \cdot 0 = 0$.

Thus, the integral takes the form

$$\frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \text{He}_j(w) e^{-w^2/2} dw = \begin{cases} 1, & j = 0, \\ 0, & j \geq 1. \end{cases}$$

Since $\mathbb{E}[\text{He}_j(z)] = 0 \forall j \geq 1$, and $\mathbb{E}[\text{He}_0(z)] = 1$.

Therefore, every term with $j \geq 1$ disappears, and only the term $j = 0$ remains:

$$y_k = \frac{e^{m+\sigma^2/2}}{k!} \left(\binom{k}{0} \sigma^{k-0} \cdot 1 \right) = \frac{e^{m+\sigma^2/2}}{k!} \cdot 1 \cdot \sigma^k.$$

Thus we obtain

$$y_k = \frac{e^{m+\sigma^2/2} \sigma^k}{k!}.$$

Exercise 4.1(c)

Now find the mean $\mathbb{E}[y]$ and variance $\mathbb{V}[y]$.

Recall

$$y(z) = \sum_{k \geq 0} y_k \text{He}_k(z), \quad y_k = \frac{e^{m+\sigma^2/2} \sigma^k}{k!}.$$

For Hermite polynomials: $\text{He}_0(z) = 1$, $\mathbb{E}[\text{He}_k(z)] = 0$ for $k \geq 1$. Substituting these in gives:

$$\begin{aligned} \mathbb{E}[y] &= \mathbb{E} \left[\sum_{k \geq 0} y_k \text{He}_k(z) \right] = \sum_{k \geq 0} y_k \mathbb{E}[\text{He}_k(z)] \\ &= y_0 \mathbb{E}[\text{He}_0(z)] + \sum_{k \geq 1} y_k \mathbb{E}[\text{He}_k(z)] y_0 \cdot 1 + \sum_{k \geq 1} y_k \cdot 0 y_0. \\ &= y_0. \end{aligned}$$

In order to calculate the mean, we only need the coefficient y_0 .

From our formula for y_k , for $k = 0$ we have

$$y_0 = \frac{e^{m+\sigma^2/2} \sigma^0}{0!} = e^{m+\sigma^2/2}.$$

Thus

$$\mathbb{E}[y] = e^{m+\sigma^2/2}.$$

Using the definition

$$\mathbb{V}[y] = \mathbb{E}[(y - \mathbb{E}[y])^2],$$

and applying $\mathbb{E}[y] = y_0$, we get

$$y(z) = y_0 \text{He}_0(z) + \sum_{k \geq 1} y_k \text{He}_k(z).$$

As $\text{He}_0(z) = 1$, we have

$$y(z) - \mathbb{E}[y] = y(z) - y_0 = \sum_{k \geq 1} y_k \text{He}_k(z).$$

Squaring both sides gives:

$$(y(z) - \mathbb{E}[y])^2 = \left(\sum_{k \geq 1} y_k \text{He}_k(z) \right)^2.$$

We can rewrite the square as a double sum:

$$\left(\sum_{k \geq 1} y_k \text{He}_k(z) \right)^2 = \sum_{k \geq 1} \sum_{\lambda \geq 1} y_k y_\lambda \text{He}_k(z) \text{He}_\lambda(z).$$

Taking expectations gives:

$$\begin{aligned} \mathbb{V}[y] &= \mathbb{E}[(y - \mathbb{E}[y])^2] \\ &= \mathbb{E} \left[\sum_{k \geq 1} \sum_{\lambda \geq 1} y_k y_\lambda \text{He}_k(z) \text{He}_\lambda(z) \right] \\ &= \sum_{k \geq 1} \sum_{\lambda \geq 1} y_k y_\lambda \mathbb{E}[\text{He}_k(z) \text{He}_\lambda(z)]. \end{aligned}$$

By the orthogonality relation:

$$\mathbb{E}[\text{He}_k(z) \text{He}_\lambda(z)] = k! \delta_{k\lambda}.$$

Therefore all cross terms with $k \neq \lambda$ vanish, leaving

$$\mathbb{V}[y] = \sum_{k \geq 1} y_k^2 \mathbb{E}[\text{He}_k(z)^2] = \sum_{k \geq 1} y_k^2 k!$$

We have

$$y_k = \frac{e^{m+\sigma^2/2} \sigma^k}{k!}.$$

Since

$$y_k^2 = \left(\frac{e^{m+\sigma^2/2} \sigma^k}{k!} \right)^2 = e^{2m+\sigma^2} \frac{\sigma^{2k}}{(k!)^2}.$$

Thus

$$\begin{aligned} \mathbb{V}[y] &= \sum_{k=1}^{\infty} y_k^2 k! \\ &= \sum_{k=1}^{\infty} e^{2m+\sigma^2} \frac{\sigma^{2k}}{(k!)^2} k! \\ &= e^{2m+\sigma^2} \sum_{k=1}^{\infty} \frac{\sigma^{2k}}{k!}. \end{aligned}$$

The power series of the exponential function is given by

$$e^{\sigma^2} = \sum_{k=0}^{\infty} \frac{\sigma^{2k}}{k!}.$$

So

$$\sum_{k=1}^{\infty} \frac{\sigma^{2k}}{k!} = e^{\sigma^2} - \frac{\sigma^0}{0!} = e^{\sigma^2} - 1.$$

In turn,

$$\mathbb{V}[y] = e^{2m+\sigma^2} (e^{\sigma^2} - 1).$$

And so it shows that

$$\mathbb{E}[y] = e^{m+\sigma^2/2}, \quad \mathbb{V}[y] = e^{2m+\sigma^2} (e^{\sigma^2} - 1).$$

Exercise 4.2.

In this exercise, we will perform an approximation of the exercise above, truncating the Hermite PCE at order $K = 10$ and using only a finite number of samples. From exercise 4.1., we know that

$$y(z) = e^{m+\sigma z}, \quad z \sim \mathcal{N}(0, 1)$$

has coefficients in the form

$$y_k = \frac{1}{k!} \mathbb{E}[y \text{He}_k(z)].$$

From 4.1(a), we know that

$$y_k = \frac{1}{\mathbb{E}[\text{He}_k(z)^2]} \mathbb{E}[y \text{He}_k(z)], \quad \text{with} \quad \mathbb{E}[\text{He}_k(z)^2] = k!$$

This means we want to approximate the expectation

$$\mathbb{E}[y \text{He}_k(z)].$$

We draw independent samples from $z_i \sim \mathcal{N}(0, 1)$, $i = 1, \dots, N$ and set $y_i := e^{m+\sigma z_i}$ then by the law of large numbers,

$$\frac{1}{N} \sum_{i=1}^N y_i \text{He}_k(z_i) \rightarrow \mathbb{E}[y \text{He}_k(z)] \quad \text{as } N \rightarrow \infty.$$

Thus we get the Monte Carlo estimator

$$\hat{\mathbb{E}}^{(N)}[y \text{He}_k(z)] := \frac{1}{N} \sum_{i=1}^N y_i \text{He}_k(z_i)$$

Substituting for y_k gives

$$\hat{y}_k^{(N)} := \frac{1}{k!} \hat{\mathbb{E}}^{(N)}[y \text{He}_k(z)] = \frac{1}{k!} \frac{1}{N} \sum_{i=1}^N y_i \text{He}_k(z_i), \quad y_i = e^{m+\sigma z_i}$$

```
In [51]: import numpy as np
import matplotlib.pyplot as plt
from math import factorial
from dataclasses import dataclass
from typing import Callable

np.random.seed(2007942)
```

Using the Hermite recurrence relation, we know that

$$\text{He}_0(z) = 1$$

$$\text{He}_1(z) = z$$

$$\text{He}_{k+1}(z) = z \text{He}_k(z) - k \text{He}_{k-1}(z)$$

I opted for a dataclass implementation because it provides a clean way to use different sampling distributions, orthogonal bases, forward models and norms. These tasks are mainly doing estimations of different arrangements of data, rather than creating and manipulating different states, hence why the typical OOP approach wasn't used. It also means there is no need to have an initialiser or the need to use `@staticmethod` either.

```
In [52]: def hermite_basis(z, k):
# recurrence relation for probabilists' Hermite polynomials
z = np.asarray(z, dtype=float)
N = z.size

He = np.empty((k+1, N))
He[0] = 1.0

if k >= 1:
    He[1] = z
    for n in range(1, k):
        He[n+1] = z * He[n] - n * He[n-1]

    return He

def estimate_coefficients(sampler, forward_model, basis_evaluator, basis_norms, K, N):
    assert isinstance(K, int) and K >= 0, "Polynomial order K must be non-negative."
    assert N > 0, "Sample size N must be positive."
```

```

z = np.asarray(sampler(N), dtype=float)
assert z.ndim == 1, "Sampler must return a 1D array."

y = np.asarray(forward_model(z), dtype=float)
assert y.shape == z.shape, "Forward model output must match sample shape."

psi = np.asarray(basis_evaluator(z, K))
assert psi.shape == (K+1, z.size), \
    f"Basis evaluator must return shape {(K+1, z.size)}, got {psi.shape}"

E_y_psi = np.mean(psi * y, axis=1)
coeffs = E_y_psi / np.asarray(basis_norms, dtype=float)

return coeffs, z, y

@dataclass
class PCEExperiment:
    order: int
    sampler: Callable[[int], np.ndarray]
    forward: Callable[[np.ndarray], np.ndarray]
    basis: Callable[[np.ndarray, int], np.ndarray]
    norms: np.ndarray

    def estimate(self, N):
        return estimate_coefficients(
            sampler=self.sampler,
            forward_model=self.forward,
            basis_evaluator=self.basis,
            basis_norms=self.norms,
            K=self.order,
            N=N,
        )

```

Each coefficient has the Monte Carlo form

$$\hat{y}_k^{(N)} = \frac{1}{k!} \frac{1}{N} \sum_{i=1}^N y_i, \text{He}_k(z_i),$$

so by the central limit theorem we expect

$$\hat{y}_k^{(N)} - y_k = \mathcal{O}(N^{-1/2})$$

as $N \rightarrow \infty$. In other words, both the absolute error

$$|y_k - \hat{y}_k^{(N)}|$$

and the relative error

$$\frac{|y_k - \hat{y}_k^{(N)}|}{|y_k|}$$

should decay like $N^{-1/2}$.

```

In [53]: def plot_pce_convergence(experiment, N_values, exact_coeffs=None,
    modes=(0, 1, 2, 3), figsize=(10, 4)):

    abs_errors = {k: [] for k in modes}
    rel_errors = {k: [] for k in modes}

    # Run estimation for each N
    for i, N in enumerate(N_values):
        coeffs_hat, _, _ = experiment.estimate(N=N)

        # If exact coefficients unknown: use the largest N as "truth"
        if exact_coeffs is None:
            exact_coeffs = coeffs_hat

        for k in modes:
            err = coeffs_hat[k] - exact_coeffs[k]
            abs_errors[k].append(abs(err))
            rel_errors[k].append(abs(err) / abs(exact_coeffs[k]))

    # Plot
    fig, axes = plt.subplots(1, 2, figsize=figsize)

    ax = axes[0]

```

```

for k in modes:
    ax.loglog(N_values, abs_errors[k], marker="o", label=f"k={k}")
ax.set_xlabel("N (sample size)")
ax.set_ylabel(r"$|y_k - \hat{y}_k^{(N)}|$")
ax.set_title("2007942 - Absolute error")
ax.legend()

ax = axes[1]
for k in modes:
    ax.loglog(N_values, rel_errors[k], marker="o", label=f"k={k}")
ax.set_xlabel("N (sample size)")
ax.set_ylabel(r"$|y_k - \hat{y}_k^{(N)}| / |y_k|$")
ax.set_title("2007942 - Relative error")
ax.legend()

plt.tight_layout()
plt.show()

```

For the validation step we generated new, independent samples z^* and compared

- the exact quantity $y^* = \exp(m + \sigma z^*)$
- the truncated PCE reconstruction

$$\tilde{y}^* = \sum_{k=0}^K \hat{y}_k^{(N)}, \text{He}_k(z^*)$$

```

In [54]: def plot_histogram_validation(experiment, coeffs_hat, basis, K,
    N_val=50000, bins=120, m=None, sigma=None):
    """
    Compare empirical distributions of
    y = forward(z) vs truncated PCE reconstruction.
    """
    z_val = np.random.randn(N_val)
    y_true = experiment.forward(z_val)

    Phi_val = basis(z_val, K)
    y_pce = coeffs_hat @ Phi_val

    plt.figure(figsize=(8,4))
    plt.hist(y_true, bins=bins, density=True, alpha=0.5, label="Exact y")
    plt.hist(y_pce, bins=bins, density=True, alpha=0.5, label="PCE reconstruction")

    if m is not None and sigma is not None:
        # log-normal PDF for reference
        y_grid = np.linspace(y_true.min(), y_true.max(), 1000)
        pdf = (1/(y_grid*sigma*np.sqrt(2*np.pi))) * np.exp(-(np.log(y_grid)-m)**2/(2*sigma**2))
        plt.plot(y_grid, pdf, "k--", label="log-normal PDF")

    plt.xlabel("y")
    plt.ylabel("density")
    plt.legend()
    plt.title("2007942 - Distribution comparison")
    plt.tight_layout()
    plt.show()

def exact_hermite_coeffs(m, sigma, K):
    prefactor = np.exp(m + 0.5*sigma**2)
    return np.array([prefactor * sigma**n / factorial(n) for n in range(K+1)])

def exact_log_normal_moments(m, sigma):
    mean = np.exp(m + 0.5*sigma**2)
    var = np.exp(2*m + sigma**2) * (np.exp(sigma**2) - 1)
    return mean, var

def pce_moments(coeffs_hat):
    mean = coeffs_hat[0]
    var = sum((coeffs_hat[k]**2) * factorial(k) for k in range(1, len(coeffs_hat)))
    return mean, var

```

We run the simulation:

```

In [55]: m = 0.5
sigma = 0.4
K = 10

hermite_exp = PCEExperiment(
    order=K,
    sampler=Lambda N: np.random.randn(N),

```

```

forward=lambda z: np.exp(m + sigma*z),
basis=hermite_basis,
norms=np.array([factorial(k) for k in range(K+1)], dtype=float),
)

y_exact = exact_hermite_coeffs(m, sigma, K)
mean_exact, var_exact = exact_log_normal_moments(m, sigma)

coeffs_hat, z_samples, y_samples = hermite_exp.estimate(N=20000)
coeffs_hat[:5]

N_values = [500, 1000, 2000, 5000, 10000, 20000]
plot_pce_convergence(
    experiment=hermite_exp,
    N_values=N_values,
    exact_coeffs=y_exact,
    modes=(0,1,2,3)
)

fig1 = r"Figure 1. Plots for the absolute and relative errors as functions of N to show the anticipated
plt.figtext(0.5, -0.1, fig1, wrap=True, ha='center', fontsize=12)

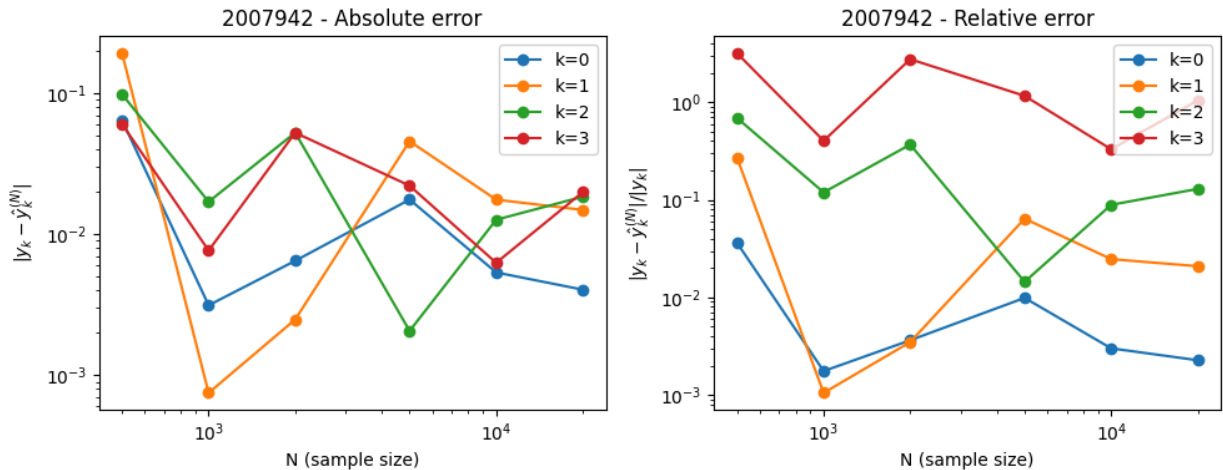
plot_histogram_validation(
    experiment=hermite_exp,
    coeffs_hat=coeffs_hat,
    basis=hermite_basis,
    K=K,
    N_val=50000,
    m=m,
    sigma=sigma,
)

fig2 = r"Figure 2. Histogram comparison between the true model output and the PCE reconstruction, along
plt.figtext(0.5, -0.1, fig2, wrap=True, ha='center', fontsize=12)

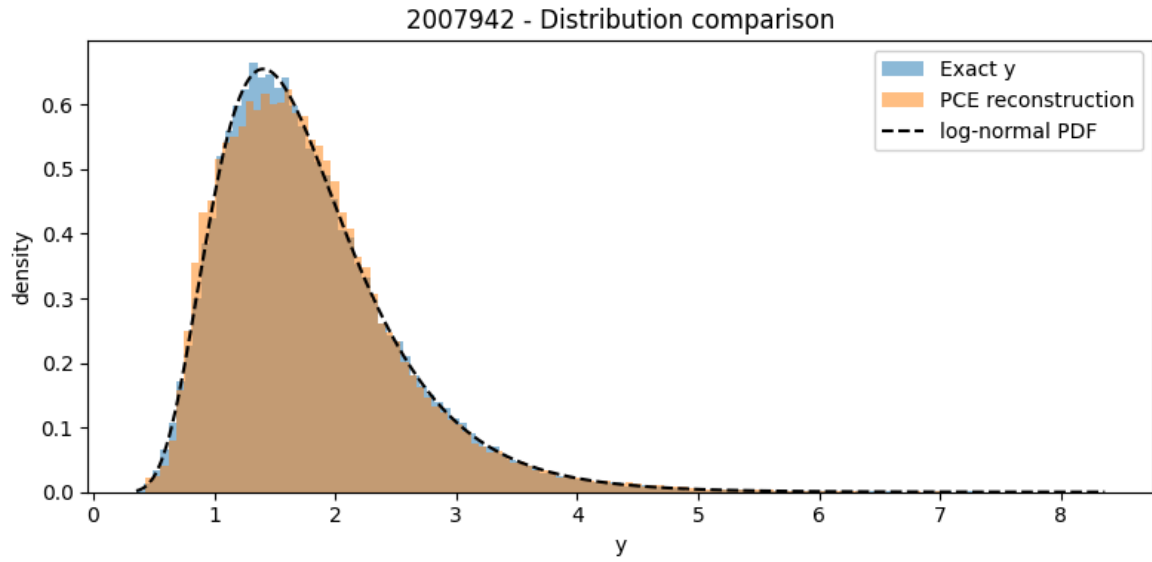
mean_pce, var_pce = pce_moments(coeffs_hat)

print(f"Exact mean: {mean_exact:.6f}")
print(f"PCE mean : {mean_pce:.6f}")
print()
print(f"Exact std : {np.sqrt(var_exact):.6f}")
print(f"PCE std : {np.sqrt(var_pce):.6f}")

```



<Figure size 640x480 with 0 Axes>



Exact mean: 1.786038
PCE mean : 1.787877

Exact std : 0.743968
PCE std : 0.753991
<Figure size 640x480 with 0 Axes>

As N increases, the errors for $k = 0, 1, 2, 3$ generally decrease, albeit nonmonotonically as a result of sampling. The lowest order coefficient y_0 is the most accurately and stably estimated as it's the mean (model output expectation), with absolute and relative errors dropping quickly as N grows. Higher order coefficients, especially $k = 2$ and $k = 3$, show larger fluctuations, which is expected because the Hermite polynomials have greater magnitude with smaller y_k coefficients. This means the MC variance is higher. The relative error is particularly large for $k = 3$, again because $|y_3|$ is small and even a modest absolute error leads to a large relative ratio. Overall, the behaviour is consistent with the anticipated $N^{-1/2}$ Monte Carlo convergence.

The histogram plots of y^* and $\tilde{y}^*$ show they have the same mode and similar spread. The shape of both histograms follow that of the actual log-normal PDF. The truncated Hermite PCE gives an accurate representation of the distribution of y for $K = 10$.

The exact mean and variance of the log normal variable are

$$\mathbb{E}[y] = e^{m+\sigma^2/2}, \quad \text{Var}[y] = e^{2m+\sigma^2}(e^{\sigma^2} - 1).$$

For $m = 0.5$ and $\sigma = 0.4$ this gives the mean = 1.786038 and exact standard deviation = 0.743968

Using the truncated, sample-based PCE we have approximate mean $\hat{y}_0^{(N)} \approx 1.787877$ and approximate variance

$$\sum_{k=1}^K (\hat{y}_k^{(N)})^2 \mathbb{E}[\text{He}_k(z)^2] = \sum_{k=1}^K (\hat{y}_k^{(N)})^2 k!$$

giving standard deviation ≈ 0.753991 .

The relative error in the mean is about 0.1, and the relative error in the standard deviation is about 1. These small discrepancies are consistent with the Monte Carlo approximation error and with truncation of the Hermite expansion at $K = 10$, and confirming that the PCE captures the first two moments of y well.

Exercise 4.3.

In this exercise we study a damped oscillator governed by

$$\dot{x}(t) = \begin{pmatrix} 0 & 1 \\ -\omega^2 & -\gamma \end{pmatrix} x(t)$$

$$x(0) = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

where we take $\omega = 1$ and model the damping coefficient γ as

$$\gamma = \frac{1}{4} + \frac{1}{4}u,$$

with $u \sim U([-1, 1])$. Our quantity of interest is the first component of $x(5)$, approximated using the Legendre polynomial chaos expansion

$$Q = \sum_{k \geq 0} Q_k L_k(u).$$

This is truncated at degree $K = 10$, with estimated coefficients from Monte Carlo sampling.

This exercise looks to generate N independent samples of u , numerically solve the ODE for each sample from $t = 0$ to $t = 5$, record the resulting values of Q , and approximate the Legendre coefficients using

$$\hat{Q}_k^{(N)} \approx Q_k = (2k + 1) \mathbb{E}[Q L_k(u)].$$

```
In [56]: def legendre_basis(u, K):
    u = np.asarray(u, dtype=float)
    N = u.size

    L = np.empty((K+1, N))
    L[0] = 1.0

    if K >= 1:
        L[1] = u
        for n in range(1, K):
            L[n+1] = ((2*n+1)*u*L[n] - n*L[n-1]) / (n+1)

    return L

legendre_norms = np.array([1/(2*k+1) for k in range(K+1)])
sampler_L = lambda N: np.random.uniform(-1,1,size=N)
```

For the RK4A implementation, I adapted the code to use vectorised operations instead of passing a function as an argument. The original implementation isn't ideal in this scenario; as N grows, so does the number of function calls. With the original RK4A method from Assignment 1, my plots took ~6 minutes. Through vectorised operations (leveraging BLAS and C optimised code), execution speed went down to around ~1 second.

```
In [57]: def forward_ode(u_samples, n_steps=200):

    u = np.asarray(u_samples, dtype=float)
    N = u.size

    gamma = 0.25 + 0.25 * u

    t0, tT = 0.0, 5.0
    dt = (tT - t0) / n_steps

    x = np.zeros((2, N))
    x[0] = 1.0 # x1(0)
    x[1] = 2.0 # x2(0)

    for _ in range(n_steps):
        # k1
        k1_0 = x[1]
        k1_1 = -x[0] - gamma * x[1]

        # k2
        x1_0 = x[0] + 0.5 * dt * k1_0
        x1_1 = x[1] + 0.5 * dt * k1_1
        k2_0 = x1_1
        k2_1 = -x1_0 - gamma * x1_1

        # k3
        x2_0 = x[0] + 0.5 * dt * k2_0
        x2_1 = x[1] + 0.5 * dt * k2_1
        k3_0 = x2_1
        k3_1 = -x2_0 - gamma * x2_1

        # k4
        x3_0 = x[0] + dt * k3_0
        x3_1 = x[1] + dt * k3_1
        k4_0 = x3_1
        k4_1 = -x3_0 - gamma * x3_1

        x[0] += dt * (k1_0 + 2*k2_0 + 2*k3_0 + k4_0) / 6.0
        x[1] += dt * (k1_1 + 2*k2_1 + 2*k3_1 + k4_1) / 6.0

    return x[0]
```

Now we instantiate a `PCEExperiment` and plot the error curves and histograms.

```
In [58]: legendre_exp = PCEExperiment(
    order=K,
    sampler=sampler_L,
    forward=forward_ode,
    basis=legendre_basis,
    norms=legendre_norms
)
Q_hat, u_samp, Q_samples = legendre_exp.estimate(N=20000)
Q_hat[:6]

plot_pce_convergence(
    experiment=legendre_exp,
    N_values=[500,1000,2000,5000,10000,20000],
    exact_coeffs=None,
    modes=(0,1,2,3)
)

fig3 = r"Figure 3. Plots for the absolute and relative errors as functions of N to show the anticipated
plt.figtext(0.5, -0.1, fig3, wrap=True, ha='center', fontsize=12)
def plot_histogram_validation(experiment, coeffs_hat, basis, K, N_val=40000, bins=100):

    u_val = experiment.sampler(N_val)
    Q_true = experiment.forward(u_val)

    Phi = basis(u_val, K)
    Q_pce = coeffs_hat @ Phi

    plt.figure(figsize=(8,4))
    plt.hist(Q_true, bins=bins, density=True, alpha=0.5, label="Exact QoI")
    plt.hist(Q_pce, bins=bins, density=True, alpha=0.5, label="PCE approximation")

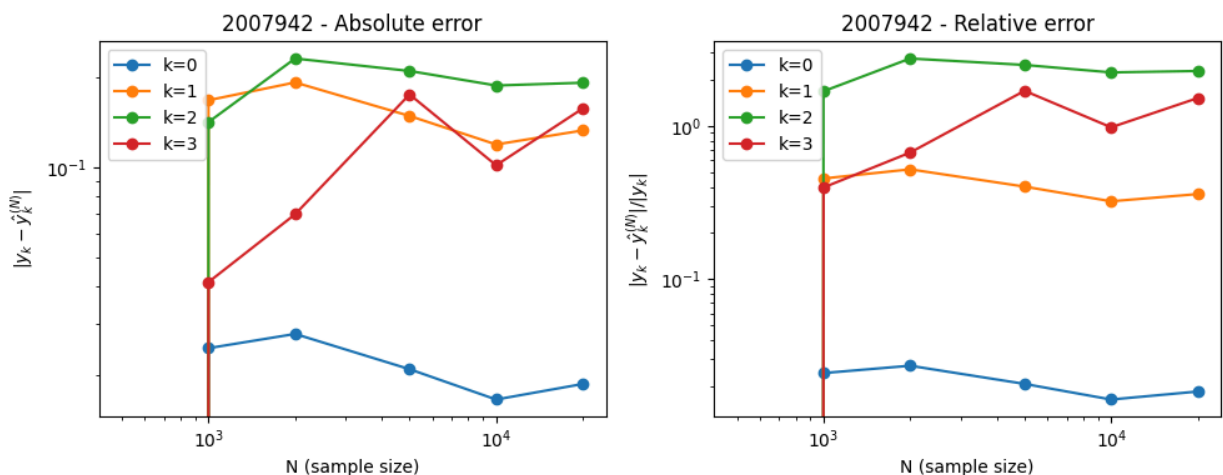
    plt.xlabel("Q = x1(5)")
    plt.ylabel("density")
    plt.legend()
    plt.title("2007942 - Histogram comparison")
    plt.tight_layout()
    plt.show()

plot_histogram_validation(
    experiment=legendre_exp,
    coeffs_hat=Q_hat,
    basis=legendre_basis,
    K=K,
    N_val=50000
)

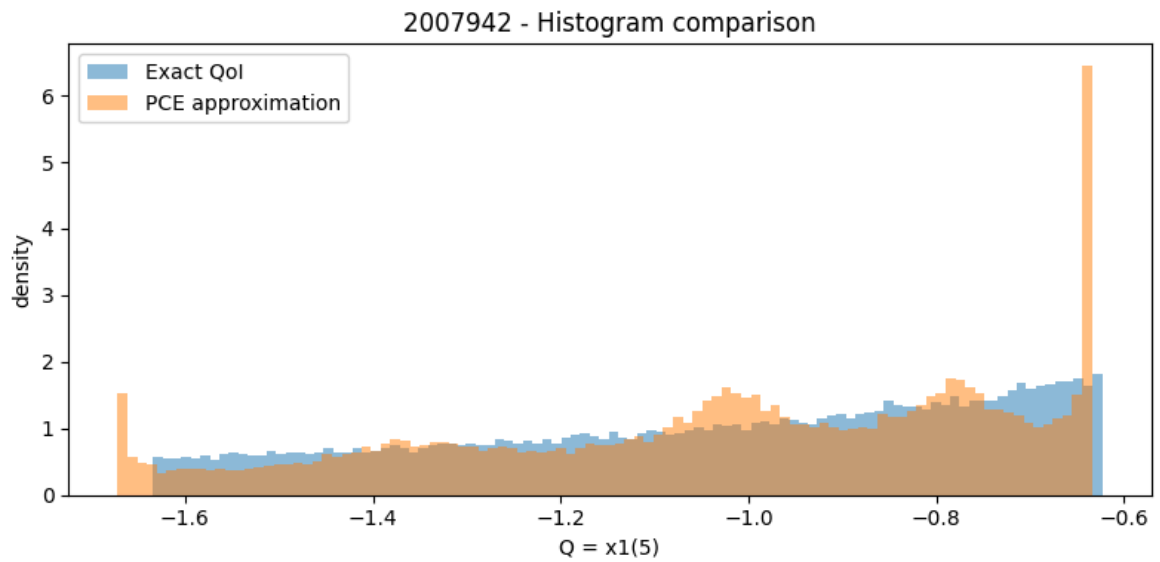
fig4 = r"Figure 4. A histogram comparison between the true model output and the PCE approximation."
plt.figtext(0.5, -0.1, fig4, wrap=True, ha='center', fontsize=12)
mean_mc = np.mean(Q_samples)
std_mc = np.std(Q_samples)

mean_pce, var_pce = pce_moments(Q_hat)

print(f"Mean MC: {mean_mc}, Stdev MC: {std_mc}")
print(f"Mean PCE:{mean_pce}, Stdev PCE: {np.sqrt(var_pce)}")
```



<Figure size 640x480 with 0 Axes>



Mean MC: -1.028988971227116, Stdev MC: 0.286800312289355
 Mean PCE: -1.028988971227116, Stdev PCE: 48.31815993015475
 <Figure size 640x480 with 0 Axes>

For the convergence plots, as N increases, there is downward trend in the errors, as we plot how estimates. For the histogram, it shows that the distributional behaviour generally matches; the density for the PCE approximation with increasing $x(5)$, and the same follows for the true histogram. Whilst the means are near identical, there is a significant difference in standard deviation.